中山大学专业硕士学位论文

基于 PyTOD 与 FlashANNS 的大规模异常检测系统优化

Optimization of PyTOD-FlashANNS for Large-scale Outlier Detection

专　　业：　工程硕士 人工智能（高性能计算方向）

学位申请人：　huangyw

指 导 教 师：　（待补充）

论文答辩委员会主席：

论文答辩委员会成员：

2026 年 3 月

论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期：　　年　　月　　日

学位论文使用授权声明

本人完全了解中山大学有关保留、使用学位论文的规定，即：学校有权保留学位论文并向国家主管部门或其指定机构送交论文的电子版和纸质版；有权将学位论文用于非赢利目的的少量复制并允许论文进入学校图书馆、院系资料室被查阅；有权将学位论文的内容编入有关数据库进行检索；可以采用复印、缩印或其他方法保存学位论文；可以为建立了馆际合作关系的兄弟高校用户提供文献传递服务和交换服务。

保密论文保密期满后，适用本声明。

学位论文作者签名：　　导师签名：

日期：　　年　　月　　日　　日期：　　年　　月　　日

摘　　要

面向金融风控、反欺诈、反洗钱与平台治理等高时效场景，异常检测系统不仅需要保持对少量高风险样本的有效识别能力，而且需要在大规模数据条件下满足严格的时延与吞吐约束。传统基于全量距离计算或精确近邻搜索的异常检测方法，在数据规模扩展至千万级乃至亿级时，往往面临计算复杂度高、显存占用大以及工程部署成本高等问题，从而难以直接支撑工业场景中的稳定运行与持续迭代。

PyTOD 通过将异常检测过程抽象为一组可复用的张量算子，为距离计算、排序、聚合与统计过程提供了统一的 GPU 执行语义；FlashANNS 则通过面向大规模近似最近邻检索的索引组织、候选筛选与高吞吐执行机制，为海量向量上的高效候选召回提供了系统基础。本文在此基础上提出 PyTOD-FlashANNS 协同优化框架，将 PyTOD 作为异常评分主干，将 FlashANNS 作为候选召回加速模块，并以统一的数据流、算子流与调度流将两者耦合为端到端的大规模异常检测系统。

围绕该框架，本文的主要工作包括四个方面。第一，面向异常检测任务构建统一的 baseline 与加速路径，对数据切分、脚本入口、指标口径与日志链路进行规范化整理，从而保证实验可复现、结果可追踪、优化可回退。第二，在系统实现层面，将 PyTOD 与 FlashANNS 进行协同集成，并在性能分析过程中识别出瓶颈主要位于 CPU 与 GPU 之间的数据搬运、候选中间结果序列化及跨阶段同步开销。针对上述问题，本文通过缩短主机侧通路、合并批量数据传输、减少不必要的主机-设备往返以及重构常驻执行路径等方式，降低了通信阻塞对端到端吞吐的影响。第三，在计算后端层面，针对原有 PyTorch 路径在部分表格型数据处理、候选组织与 GPU 数据框操作中的额外开销，本文将相关实现迁移到 RAPIDS 生态，并结合 FAISS-GPU 完成高吞吐近似检索后端的重组，以进一步提升向量处理与统计计算效率。第四，在多 GPU 场景下，本文按照“复制式多实例（每 GPU 一套 FlashANNS 管线）+ 请求级动态调度（pull/队列）+ 微批/自适应批量（近实时约束下的 bounded batching）”的推荐方案 A，对系统进行负载均衡与吞吐扩展优化，以降低跨卡归并代价并提升设备利用率，使多卡场景下的整体吞吐更接近线性增长。

本文的研究重点不在于给出单一算子的局部最优实现，而在于说明在准确率约束下，如何围绕候选召回、精排计算、主机-设备通信与多卡调度进行协同设计，从而推动 PyTOD 更适用于工业环境中的实际部署与持续运行。考虑到正式送审版仍需根据最终复核结果补齐实验图表，本文在实验章节中仅保留图表

标题、应展示的数据内容及其论证作用说明，不预先填入结果数值，以便后续在统一口径下完成定稿。

关键词：异常检测，PyTOD，FlashANNS，RAPIDS，FAISS-GPU，CPU-GPU 通信优化，多 GPU 调度，近似最近邻检索

ABSTRACT

This thesis investigates PyTOD-FlashANNS, a collaborative optimization framework for large-scale outlier detection. In industrial scenarios such as fraud detection, anti-money laundering, and platform governance, an outlier detection system must preserve detection effectiveness while satisfying stringent latency and throughput requirements. When the data scale grows to tens or hundreds of millions of samples, brute-force distance computation and exact nearest-neighbor search become difficult to sustain because of their computational complexity, memory pressure, and deployment overhead.

PyTOD provides a tensor-operator abstraction for outlier detection, enabling distance computation, sorting, aggregation, and statistical scoring to be expressed within a unified GPU execution model. FlashANNS, in contrast, focuses on high-throughput approximate nearest-neighbor retrieval and improves candidate generation through better index organization, candidate pruning, and efficient execution. This thesis integrates the two systems into a unified end-to-end pipeline in which PyTOD serves as the scoring backbone and FlashANNS serves as the candidate-retrieval accelerator.

The thesis makes four main contributions. First, it establishes a unified benchmarking and logging pipeline for both the baseline and the accelerated paths so that experiments are reproducible and optimization steps remain auditable. Second, it identifies the major system bottleneck in the communication and synchronization overhead between CPU and GPU, including intermediate serialization, host-side blocking, and repeated device transfers, and proposes a shorter and more persistent execution path to reduce such overhead. Third, it further accelerates computation by migrating selected processing stages from the original PyTorch-based path to the RAPIDS ecosystem and by reorganizing the retrieval backend around FAISS-GPU. Fourth, for multi-GPU deployment, it adopts the recommended Scheme A, namely replicated multi-instance serving with one FlashANNS pipeline per GPU, pull-based request scheduling, and bounded micro-batching under near-real-time constraints, to improve load balancing and reduce cross-device merge overhead, thereby enabling throughput scaling that approaches linear growth with the number of GPUs.

The core contribution of this work is therefore not an isolated micro-optimization, but an end-to-end orchestration strategy that jointly considers candidate retrieval, reranking, CPU-GPU communication, and multi-GPU scheduling under accuracy con-

straints. Since the final submission version still requires result verification, the experimental chapter intentionally leaves result figures and tables unfilled and only specifies what data each figure or table should present and why it is necessary for the argument.

Keywords: Outlier Detection, PyTOD, FlashANNS, RAPIDS, FAISS-GPU, CPU-GPU Communication Optimization, Multi-GPU Scheduling, Approximate Nearest Neighbor Search

目　　录

第 1 章　绪论 1
1.1　研究背景与意义 1
1.2　研究对象与核心问题 2
1.3　研究目标 2
1.4　主要创新点 3
1.5　论文组织 3
第 2 章　相关研究工作 4
2.1　距离型异常检测与大规模计算挑战 4
2.2　TOD/PyTOD 的方法学基础 4
2.3　FlashANNS 与候选召回系统 4
2.4　RAPIDS、FAISS-GPU 与工程可部署性 5
2.5　多 GPU 调度与在线推理启示 5
第 3 章　系统架构与实现 6
3.1　设计原则 6
3.2　总体架构 6
3.3　CPU-GPU 通信路径分析 7
3.4　计算后端与数据处理实现 7
3.5　多 GPU 扩展策略 7
3.6　日志与可观察性设计 8
第 4 章　关键优化过程 10
4.1　早期原型与瓶颈暴露 10
4.2　通路重构：面向 CPU-GPU 通信效率的执行链路压缩 10
4.3　计算后端优化：由 PyTorch 路径向 RAPIDS 迁移 10
4.4　候选召回后端重组：与 FAISS-GPU 的协同实现 11
4.5　精排路径与参数重整定 11
4.6　方案 A：复制式多实例、请求级动态调度与 bounded batching 12
4.7　负向路径、回退机制与方法论 12
第 5 章　实验设计与结果分析 14
5.1　实验环境与软硬件配置 14

5.2 数据集与任务设置 14
5.3 评价指标与比较原则 15
5.4 需要补齐的核心结果图表 15
5.5 结果讨论应回答的问题 17
第 6 章 全文总结与展望 19
6.1 全文总结 19
6.2 后续工作展望 19
参考文献 21
附录 A 实验脚本与日志模板 22
A.1 建议补充内容 22
附录 B 图表补充说明 23
B.1 建议补充内容 23
附录 C 缩写、变量与配置对照 24
致谢 25

符号和缩略语说明

缩写	含义
ANN	Approximate Nearest Neighbor，近似最近邻
AUC	Area Under Curve，曲线下面积
BTO	Basic Tensor Operator，基础张量算子
FAISS	Facebook AI Similarity Search
FO	Functional Operator，功能算子
GPU	Graphics Processing Unit，图形处理器
KNN	K-Nearest Neighbors，k 近邻
PR-AUC	Precision-Recall Area Under Curve
QPS	Queries Per Second，每秒查询数
RAPIDS	面向 GPU 数据科学与分析计算的开源加速生态
ROC-AUC	Receiver Operating Characteristic Area Under Curve
TOD	Tensor Operations for Outlier Detection
Top-k overlap	近似排序结果与基线排序结果的前 k 重合度
`adaptive_ratio`	多 GPU 动态分流比例策略
bounded batching	在时延约束下受控执行的微批处理策略
rerank	对粗排候选执行更高精度精排计算的步骤

第 1 章　绪论

1.1　研究背景与意义

异常检测是金融风控、反欺诈、反洗钱与平台治理等场景中的基础能力，其目标是在大规模正常样本中识别少量偏离总体分布的高风险对象。与传统监督学习问题相比，异常检测通常伴随标签稀缺、类别极度不平衡、异常形态持续演化以及业务时效要求高等现实约束，因此其研究重点不仅在于模型精度，也在于系统在真实环境中的部署效率与可扩展性。围绕异常检测的算法与系统研究已形成较为丰富的方法基础，其中 TOD/PyTOD 提供了统一张量算子抽象，经典异常检测分析则为系统问题建模提供了通用理论视角[1-2]。

在样本规模较小时，基于精确近邻搜索或全量距离矩阵的异常检测方法仍可在可接受时间内完成训练与评分；然而，当数据规模上升至千万级、亿级时，距离计算复杂度、显存压力、主机-设备通信代价以及多设备调度成本会同时成为制约系统落地的关键因素。现有 GPU 向量检索基础设施已经证明大规模相似度搜索在工程上具备可加速空间[3]。因而，如何在保持准确率约束的前提下将异常检测系统推向可复现、可部署、可扩展的工程形态，已成为该方向的重要研究课题。

正式版补齐图 1-1

图 1-1　研究问题全景图

图示内容要求：建议绘制“业务场景 - PyTOD 张量算子骨架 - FlashANNS 候选召回 - RAPIDS/FAISS-GPU 加速模块 - 多 GPU 调度 - 统一评测输出”的总览示意图。图中应标出数据输入、候选检索、精排、异常分数聚合以及日志分析五类核心模块，并用箭头标注数据流向与控制流向。

图示作用说明：图 1-1 用于在绪论中说明本文研究对象是端到端的大规模异常检测系统，而非单一算法或单一算子优化。

表 1-1　本文研究问题、技术路径与验证目标

正式版补齐表 1-1

表格内容要求：建议设置“研究问题、瓶颈来源、拟采用技术、验证指标、预期贡献”五列，明确展示 PyTOD-FlashANNS 协同、CPU-GPU 通信优化、RAPIDS 迁移及多 GPU 调度优化之间的对应关系。

表格作用说明：表 1-1 用于帮助读者快速理解本文的总体研究框架、问题分解方式与验证逻辑。

1.2　研究对象与核心问题

PyTOD 的核心价值在于统一抽象。它将多种异常检测算法分解为一组可复用的基础张量算子与功能算子，使距离、排序、聚合与统计等过程能够在 GPU 上以统一语义执行[1]。FlashANNS 的核心价值则在于高吞吐候选召回。对于海量向量数据而言，先快速召回一小部分高价值候选，再执行高精度精排，通常比对全量样本做精确比较更符合工业场景中的时延和成本约束[4]。

基于这一互补关系，本文将研究对象定义为由数据准备、索引构建、候选检索、精排计算、异常分数聚合、日志分析和多 GPU 调度组成的端到端系统。与仅关注异常评分公式或仅关注 ANN 检索效率的研究不同，本文强调在统一精度门控条件下，对候选流、算子流和调度流进行协同优化。

1.3　研究目标

本文围绕三个层面的核心目标展开。第一，在准确率约束下提升吞吐，即在 ROC-AUC、PR-AUC 及排序一致性指标不发生不可接受退化的前提下，提高异常评分系统的端到端处理能力。第二，降低系统中非必要的工程开销，尤其是 CPU 与 GPU 之间由中间序列化、频繁搬运与阶段同步引入的额外延迟。第三，在多 GPU 条件下构建具有工业可部署性的扩展策略，使系统在保持实现复杂度

可控的条件下获得接近线性的吞吐增长。

围绕上述目标，本文进一步提出四项具体任务：构建统一实验基线；实现 PyTOD 与 FlashANNS 的协同集成；完成 PyTorch 到 RAPIDS 的关键子路径迁移与 FAISS-GPU 重组；按照推荐方案 A 设计复制式多实例、多卡动态调度与 bounded batching 机制。

1.4　主要创新点

第一，本文提出了面向大规模异常检测的 PyTOD-FlashANNS 协同优化框架，将异常检测张量执行主干与近似最近邻候选召回模块纳入统一流水线，使 PyTOD 从以算法加速为中心的实现形态扩展为更适合工业场景落地的系统形态。

第二，本文在优化过程中识别出瓶颈并非仅存在于算子执行本身，而大量集中在 CPU 与 GPU 之间的数据搬运、阶段边界处的中间结果表达以及主机侧阻塞。围绕这一发现，本文以缩短主机通路、减少不必要往返、合并批量传输和重构常驻执行路径为主线，对系统进行了通信效率优化。

第三，本文将部分原本依赖 PyTorch 的处理路径迁移至 RAPIDS 生态，以提升 GPU 上表格型数据处理、候选组织与统计计算的执行效率，并与 FAISS-GPU 组合形成更稳定的高吞吐实现。

第四，本文针对多 GPU 场景提出并工程化实现推荐方案 A，即复制式多实例（每 GPU 一套 FlashANNS 管线）+ 请求级动态调度（pull/队列）+ bounded batching。在避免跨卡归并成为主瓶颈的前提下，该方案更适合在 PCIe 互连环境下追求吞吐近线性扩展。

1.5　论文组织

全文按照“问题提出 - 相关研究 - 系统设计与实现 - 关键优化过程 - 实验设计与结果说明 - 总结与展望”的逻辑展开。第二章给出与 TOD/PyTOD、FlashANNS、FAISS-GPU 及多 GPU 在线推理调度相关的研究基础；第三章说明系统架构与模块实现；第四章梳理关键优化过程及其方法论；第五章给出实验设计、图表占位与结果应展示的内容；第六章总结全文并讨论后续研究方向。

第 2 章　相关研究工作

2.1　距离型异常检测与大规模计算挑战

距离型异常检测方法通常基于样本间相对邻近关系来刻画异常程度，例如基于 k 近邻距离、局部离群因子和邻域统计的各类方法。这类方法具有较强的直观性和解释性，但其共同问题在于需要进行大量邻域搜索与距离计算。相关研究已指出，当样本规模不断增大时，全量距离矩阵既难以存储，也难以在可接受时间内完成计算[2]。

因此，大规模异常检测系统通常需要同时解决两个问题：一是通过近似搜索或候选缩减减少候选集合规模，二是通过 GPU 张量化执行、自动批处理和多设备并行降低精排计算成本。本文的系统设计正建立在这两个方向的协同之上。

2.2　TOD/PyTOD 的方法学基础

TOD 提出将异常检测算法抽象为基础张量算子与功能算子的编程模型，从而使复杂异常检测流程能够复用统一的 GPU 执行基础设施[1]。该方法不仅降低了为不同检测器分别开发 GPU 内核的工程成本，而且为后续围绕批处理、算子融合与多 GPU 支持展开系统优化提供了统一接口。

PyTOD 作为 TOD 的开源实践载体，为距离计算、排序、聚合与统计过程提供了较成熟的实现基础。本文将其视为异常评分主干，并在此基础上引入候选召回与系统层调度机制。

2.3　FlashANNS 与候选召回系统

FlashANNS 面向大规模近似最近邻检索，强调在外存或大规模索引环境中对存储访问与计算执行进行协同设计。其核心思想在于通过更合理的索引组织、候选过滤与异步执行机制提升高维向量检索吞吐，从而为需要大规模候选召回的上层应用提供系统支撑[4]。与之相关的近似最近邻检索研究也表明，候选图结构与检索路径设计会显著影响系统在高维空间中的吞吐与鲁棒性[5]。

对异常检测而言，FlashANNS 的价值不在于替代异常评分模型，而在于为后续精排提供规模可控且质量可接受的候选集合。因而，本文并不将 FlashANNS 视为独立系统，而是将其定位为 PyTOD 异常评分流程中的前置加速器。

2.4　RAPIDS、FAISS-GPU 与工程可部署性

在研究型原型向生产型系统演化的过程中，计算后端的稳定性、生态成熟度与部署可维护性具有重要意义。RAPIDS 提供了面向 GPU 的数据框、机器学习与数据科学工具链，适合处理表格型数据转换、统计计算与部分候选组织任务。FAISS-GPU 则在大规模向量相似度检索方面具有成熟实现和广泛工程实践[3]。

本文将 RAPIDS 与 FAISS-GPU 引入 PyTOD-FlashANNS，不仅是出于局部算子提速的考虑，更是为了降低自定义服务链路与分散依赖带来的工程复杂度，使系统能够将优化重心转移到更具通用价值的参数、通信和调度层面。

2.5　多 GPU 调度与在线推理启示

面向在线推理和大规模检索的系统研究表明，复制式部署、请求级动态调度与时延约束下的受控批处理是提升吞吐与设备利用率的重要手段。特别是在设备间互连带宽有限、跨卡结果归并代价较高的环境中，将完整索引复制到每张 GPU 并通过队列或拉取机制分发请求，往往比分片后再跨卡归并更具工程优势[6-9]。

本文在多 GPU 扩展中采用的推荐方案 A，正是建立在这一思路之上：用复制式多实例避免跨卡归并成为主瓶颈，用请求级动态调度应对查询难度与设备状态差异，用 bounded batching 在吞吐与时延之间保持可控平衡。

表 2-1　相关工作比较表

正式版补齐表 2-1

表格内容要求：建议设置“工作类别、代表系统、是否支持 GPU、是否支持多 GPU、是否关注通信开销、是否关注工程部署、与本文关系”七列，比较 TOD/PyTOD、FlashANNS、FAISS-GPU 相关系统与本文方案的差异。

表格作用说明：表 2-1 用于突出本文的定位在于系统级协同优化，而非提出新的异常检测算法或新的 ANN 索引。

第 3 章　系统架构与实现

3.1　设计原则

结合异常检测业务对准确率、吞吐、稳定性与可复现性的综合要求，本文将系统设计原则概括为四点：其一，模块边界清晰，使候选召回、精排计算、日志统计与调度逻辑相互解耦；其二，数据通路尽量短，尽可能减少 CPU 与 GPU 之间不必要的数据搬运与中间序列化；其三，实验口径统一，使任意一次优化都能够在同一输入、同一指标与同一日志规范下进行真实比较；其四，多 GPU 扩展优先选择实现复杂度可控且收益可解释的方案。

3.2　总体架构

PyTOD-FlashANNS 的总体架构可划分为数据层、执行层与分析层。数据层负责样本标准化、训练集与查询集划分以及标签管理；执行层负责索引构建、请求分批、候选检索、精排与异常分数聚合；分析层负责吞吐统计、准确率评估、显存记录与日志汇总。

在执行层内部，系统包含 baseline 路径与 accelerated 路径。前者保留 PyTOD 的原始或接近原始评分逻辑，用于建立精度与吞吐基线；后者在候选召回阶段引入 FlashANNS/FAISS-GPU，并在精排与调度阶段引入 RAPIDS、通信优化与多 GPU 机制。

正式版补齐图 3-1

图 3-1　PyTOD-FlashANNS 端到端执行架构图

图示内容要求：建议采用自左向右的数据流形式展示输入数据、特征准备、索引构建、候选检索、RAPIDS 处理、PyTOD 精排、异常分数聚合和统一评测。图中应使用不同颜色或虚线框区分 PyTOD 主干、FlashANNS/FAISS-GPU 候选召回模块以及日志分析模块。

图示作用说明：图 3-1 用于说明系统模块边界、数据流向和加速模块之间的分工关系。

3.3　CPU-GPU 通信路径分析

在系统实现过程中，本文发现端到端性能瓶颈并不主要体现在某一单独算子的执行时间，而主要来自 CPU 与 GPU 之间的通信和阶段同步成本。具体而言，瓶颈表现为三个方面：一是候选中间结果在阶段边界处被重复表达、重复序列化和重复传输；二是主机侧控制逻辑导致 GPU 在等待候选或等待精排输入时出现空转；三是小批量、细粒度调用造成主机-设备往返过于频繁，削弱了 GPU 对批量并行的利用优势。

基于上述分析，本文在系统设计中采用“最小必要表达”的接口原则，仅在模块边界上传递候选索引、必要分数与元信息；同时通过批量合并、常驻执行路径和更紧凑的数据组织方式降低通信阻塞。该思路也是后续迁移到 RAPIDS 以及设计 bounded batching 的重要前提。

3.4　计算后端与数据处理实现

在候选组织、统计处理与部分表格型数据转换阶段，传统 PyTorch 路径虽然能够完成张量计算，但在 DataFrame 风格处理、索引型中间结果管理以及与检索后端衔接时存在额外开销。为此，本文将相关处理阶段迁移到 RAPIDS 生态，以利用其在 GPU 数据框与并行数据处理方面的优势。

与此同时，系统以 FAISS-GPU 作为近似最近邻检索主干，与 FlashANNS 的高吞吐候选召回思想形成互补[3-4]。通过这一组合，候选检索、数据组织与精排输入准备可以在统一的 GPU 加速路径上完成，从而减少主机参与度并提高工程可部署性。

3.5　多 GPU 扩展策略

在多 GPU 场景下，本文优先采用推荐方案 A，即在每张 GPU 上复制一套完整的 FlashANNS/FAISS-GPU 管线，并通过请求级 pull/队列调度向各设备分发批次。与分片索引方案相比，该方式虽然需要更多显存，但能够显著减少跨卡广播与归并带来的额外复杂度，更适合在 PCIe 互连环境下追求吞吐扩展。

为了兼顾吞吐与实时性，系统在请求调度层引入 bounded batching 机制。其

核心思想是在满足近实时约束的前提下，以受控方式将请求聚合为微批处理，并根据运行时设备表现动态调整各 GPU 的拉取节奏和工作比例，从而减轻长尾批次对整体完成时间的影响。

正式版补齐图 3-2

图 3-2　多 GPU 复制式多实例与动态调度示意图

图示内容要求：建议展示每张 GPU 各自持有完整候选召回与精排管线的结构，并在图中标出全局请求队列、设备拉取过程、bounded batching 形成方式以及主控进程的结果汇总位置。

图示作用说明：图 3-2 用于说明方案 A 如何通过避免跨卡归并、引入请求级动态调度和 bounded batching 来提升多卡吞吐。

3.6　日志与可观察性设计

为保证系统优化结论具有可解释性，本文要求每次实验至少记录索引构建时间、批次查询时间、候选召回时间、精排时间、总耗时、吞吐、显存占用、AUC 指标、排序一致性指标以及异常批次信息。统一的日志链路不仅支持性能定位，也为负向路径回退和最终送审前的结果复核提供了基础。

表 3-1　关键日志字段与用途说明

正式版补齐表 3-1

表格内容要求：建议按“字段名称、记录阶段、单位、用途、是否进入最终图表”

五列整理，包括 `build_time`、`query_time`、`rerank_time`、`qps`、`roc_auc`、`pr_auc`、`topk_overlap`、`gpu_mem_peak` 等核心字段。

表格作用说明：表 3-1 用于说明本文为何能够对性能收益来源进行分阶段解释，并支撑后文的图表复核。

第 4 章　关键优化过程

4.1　早期原型与瓶颈暴露

系统最初以“先跑通”为目标，允许通过文件或松耦合服务在模块之间交换数据。该方式虽然能够较快验证 PyTOD 与候选召回模块之间的基本可行性，但也迅速暴露出显著的系统瓶颈，即主机侧阻塞时间长、中间结果表达冗余、设备等待严重以及调试路径过长。

原型阶段的重要价值不在于性能，而在于帮助研究者识别真正需要工程化重构的位置。本文后续的若干收益并非直接来自更复杂的算子设计，而是来自对这一路径性瓶颈的缩短与消除。

4.2　通路重构：面向 CPU-GPU 通信效率的执行链路压缩

在识别到系统瓶颈主要集中于 CPU-GPU 通信后，本文对候选准备、结果消费和精排输入通路进行了重构。其核心目标并非引入复杂的新中间件，而是尽可能减少不必要的主机参与：一方面，减少候选集合在不同阶段之间的重复序列化和格式转换；另一方面，将若干高频、小粒度的主机-设备交互合并为更具吞吐优势的批量传输。

这一阶段的重构虽然没有改变异常评分公式本身，却显著提升了系统的可观察性与可维护性，为后续将处理路径迁移到 RAPIDS 并与 FAISS-GPU 进一步耦合奠定了基础。

4.3　计算后端优化：由 PyTorch 路径向 RAPIDS 迁移

在部分数据整理、候选组织和统计计算阶段，原有 PyTorch 路径并不能始终提供最优的端到端效率。特别是在需要进行 DataFrame 风格过滤、批量拼接、索引映射与中间统计时，PyTorch 方案往往引入额外的格式转换与主机侧辅助逻辑。

针对这一问题，本文将相关路径迁移至 RAPIDS 生态，通过 GPU 数据框与面向分析计算的并行实现减少中间格式切换，并使候选组织与统计处理更紧密地留在 GPU 上执行。该优化的学术意义在于说明，异常检测系统的性能提升并不只依赖于张量核函数本身，也依赖于整个数据处理链路的后端选择。

4.4　候选召回后端重组：与 FAISS-GPU 的协同实现

在通信路径与数据组织相对稳定后，本文进一步将候选召回主干重组为与 FAISS-GPU 更紧密协同的实现。该步骤的意义不只在于更快的检索速度，更在于通过成熟的 GPU 检索基础设施降低自定义服务的维护成本，使系统能够将研究重心转移到参数选择、精度门控与多卡调度上。

在这一阶段，PyTOD 作为异常评分主干与候选召回模块之间的接口被进一步简化，候选结果以更紧凑的表达形式进入精排阶段，从而降低后续计算的组织成本。

4.5　精排路径与参数重整定

ANN 路径的核心优势在于通过近似候选缩减显著减少精排负载，但候选数量设置过小会导致异常分数偏离基线，设置过大又会吞噬候选召回节省下来的时间。因此，索引参数与精排预算必须在完整流水线中联合考察，而不能孤立地对某一阶段进行局部最优调参。

本文围绕 `nlist`、`nprobe`、候选预算、批量大小与精排组织方式进行了系统搜索，并将满足准确率门控条件的参数组合作为后续多 GPU 调度优化的输入基础。

正式版补齐图 4-1

图 4-1　参数搜索热力图或分组柱状图

图示内容要求：建议横轴展示 `nlist`，纵轴展示 `nprobe` 或候选预算，颜色或柱高展示 QPS；同时在单元格或图例中标注 ROC-AUC、PR-AUC 或 Top-k overlap，并明确标出满足准确率门控条件的推荐点。

图示作用说明：图 4-1 用于说明参数选择是基于完整流水线下的量化搜索，而非经验式试错。

4.6 方案 A：复制式多实例、请求级动态调度与 bounded batching

在多 GPU 条件下，静态平均切分虽然实现简单，但容易受到设备差异、查询难度差异和长尾批次的影响，导致某些 GPU 提前空闲而另一些 GPU 仍在处理尾部任务。为解决这一问题，本文采用推荐方案 A：在每张 GPU 上部署完整副本，通过全局请求队列进行 pull 式任务分配，并在近实时约束下形成 bounded batching。

该方案的关键优势在于避免跨卡归并成为扩展瓶颈。由于每张 GPU 都拥有完整索引和完整处理管线，查询可直接在单卡上完成主要工作，仅需在主控进程进行轻量级结果汇总。与此同时，请求级动态调度能够根据运行时吞吐表现调整设备负载，而 bounded batching 则使系统能够在吞吐和时延之间保持可控平衡。

正式版补齐图 4-2

图 4-2　静态切分与方案 A 动态调度对比图

图示内容要求：建议使用双时间轴或甘特图展示 GPU0 与 GPU1 在静态平均切分和 pull/队列调度两种策略下的批次完成情况，并在图旁补充总耗时、尾部差值和平均设备利用率统计框。

图示作用说明：图 4-2 用于直观说明动态调度与 bounded batching 如何缓解多卡拖尾并提升整体吞吐。

4.7 负向路径、回退机制与方法论

本文保留了若干负向优化路径，例如过度激进的融合精排、额外复杂的两阶段预算自适应以及缺乏收益支撑的全链路流水化尝试。保留这些结果的原因在于，性能优化研究若只展示成功案例，往往难以说明系统瓶颈的真实位置与优化边界。

基于“先验证、再固化；无收益、即回退”的原则，本文将正向收益与负向收

益统一纳入日志和结果复核框架之中，从而使工程谨慎性成为研究方法的一部分。

表 4-1　正向路径与负向路径对照表

正式版补齐表 4-1

表格内容要求：建议设置“优化路径、目标瓶颈、关键变更、是否保留、预期收益、实际表现、失败或保留原因”七列，对通信优化、RAPIDS 迁移、FAISS-GPU 重组及多 GPU 方案 A 等路径进行并列说明。

表格作用说明：表 4-1 用于证明本文具有完整的验证闭环，而非仅展示局部成功结果。

第 5 章　实验设计与结果分析

5.1　实验环境与软硬件配置

正式版应在统一脚本下补齐软硬件环境表，明确记录 GPU 型号、显存、CUDA 版本、驱动版本、Python 版本、RAPIDS 版本、FAISS 版本以及关键依赖来源。对于跨阶段结果，应明确说明是否属于完全同口径对比。与统一脚本、参数配置和日志复核相关的材料在本文内部记录中统一管理[10]。

表 5-1　实验环境配置表

正式版补齐表 5-1

表格内容要求：建议列出硬件平台、GPU 型号与数量、驱动/CUDA 版本、Python 环境、PyTorch 版本、RAPIDS 版本、FAISS 版本以及日志脚本入口。若涉及不同阶段环境变化，应单独注明。

表格作用说明：表 5-1 用于保证结果解释建立在清晰可复现的环境前提之上。

5.2　数据集与任务设置

正式版应区分真实金融风控数据与规模扩展数据，明确给出样本规模、特征维度、异常比例、训练集、索引库与查询集划分方式及脱敏说明。所有 baseline 与加速路径应使用相同的数据切分，以保证结果具备可比性。

表格内容要求：建议列出数据集名称、样本规模、特征维度、异常比例、训练、索引、查询划分方式、是否脱敏以及使用目的。若某些数据仅用于扩展性测试，也应在备注列说明。

表格作用说明：表 5-2 用于明确本文结论适用的数据范围与业务场景边界。

表 5-2　数据集与任务设置表

正式版补齐表 5-2

5.3　评价指标与比较原则

除 ROC-AUC 与 PR-AUC 外，正式版还应补充 Top-k overlap、Spearman 相关系数、P@k、R@k、平均时延、尾延迟及显存峰值等指标。原因在于，ANN 路径可能在全局 AUC 上变化不大，但在高风险样本排序或资源占用上呈现显著差异，单一指标不足以完整反映系统行为。

比较原则上，本文坚持先报告完全同口径的结果，再对跨后端、跨路径或跨阶段的结果进行单独说明，避免将通信重构收益、后端替换收益和参数重整定收益混为一谈。

5.4　需要补齐的核心结果图表

正式版补齐图 5-1

图 5-1　分阶段收益树或瀑布图

图示内容要求：应从统一 baseline 出发，依次叠加通信路径优化、RAPIDS 迁移、FAISS-GPU 协同重组以及多 GPU 方案 A 调度，并展示每一阶段的累计吞吐变化或总时长下降幅度。

图示作用说明：图 5-1 用于将多个阶段性收益串联为清晰叙事线，区分系统重构收益与参数、调度收益。

表 5-3　核心结果总表

正式版补齐表 5-3

表格内容要求：应至少包含“阶段名称、关键变更、吞吐或总时长、相对提升、ROC-AUC、PR-AUC、Top-*k* overlap、显存峰值、是否同口径、备注”十列。该表不应预先填数，待最终复核统一填写。

表格作用说明：表 5-3 用于支撑摘要、结论与答辩展示，是全文最重要的结果总表。

正式版补齐图 5-2

图 5-2　CPU-GPU 通信瓶颈变化图

图示内容要求：应展示优化前后主机侧准备、候选传输、GPU 计算、结果汇总等阶段的时间占比变化，可采用堆叠条形图或阶段占比图。

图示作用说明：图 5-2 用于证明性能提升的重要来源在于通信与同步开销下降，而不仅是算子局部加速。

图示内容要求：应展示两种后端在候选组织、统计处理或数据准备阶段的吞吐、耗时和显存占用差异，并在图注明确比较的是哪一子流程。

图示作用说明：图 5-3 用于说明将部分实现从 PyTorch 迁移到 RAPIDS 的实际系统价值。

图示内容要求：应展示 1 卡、2 卡及更高卡数条件下的总 QPS 或总耗时变化，并同时给出理想线性扩展曲线作为参照。若采用方案 A，还应单独给出静态切分与 pull/队列调度的对比。

正式版补齐图 5-3

图 5-3　PyTorch 路径与 RAPIDS 路径对比图

正式版补齐图 5-4

图 5-4　单卡与多卡吞吐扩展图

图示作用说明：图 5-4 用于验证复制式多实例与请求级动态调度在多 GPU 场景下带来的扩展收益。

5.5　结果讨论应回答的问题

正式版结果讨论应重点回答三个问题。第一，性能收益分别来自通信路径压缩、RAPIDS 迁移、FAISS-GPU 协同重组以及多 GPU 调度的哪些环节。第二，随着优化推进，系统瓶颈如何从主机侧通信迁移到参数配置、精排预算与多卡拖尾。第三，在准确率门控条件下，系统的吞吐提升是否具有可持续性与可迁移性。

结果讨论不应仅罗列倍数，而应在图表基础上解释瓶颈迁移与收益边界。对于无法完全同口径比较的阶段性结果，应明确注明其比较条件与适用范围。

表格内容要求：建议设置“威胁来源、可能影响、是否已控制、缓解策略、后续工作”五列，覆盖环境差异、随机种子、日志清洗、异常批次剔除规则及真实业务数据不可公开等问题。

表格作用说明：表 5-4 用于增强论文的学术严谨性，并提前回应评审中可能提出

表 5-4　有效性威胁与缓解策略

正式版补齐表 5-4

的有效性问题。

第 6 章　全文总结与展望

6.1　全文总结

本文围绕“基于 PyTOD 与 FlashANNS 的大规模异常检测系统优化”这一主题，完成了从问题定义、研究定位、系统设计到实验规划的整体论证。研究表明，PyTOD 与 FlashANNS 具有天然互补性：前者提供异常评分所需的统一张量算子骨架，后者提供海量候选检索所需的高吞吐召回能力。只有将两者置于同一端到端执行链路中联合优化，才能真正推动异常检测系统走向工业可部署形态。

本文进一步强调，系统性能瓶颈并不总是来自算子执行本身。通过对候选通路、主机-设备通信、后端生态选择与多 GPU 调度的系统性分析，本文说明了 PyTOD 向更高工业适用性演进的关键路径：一是识别并降低 CPU-GPU 通信开销，二是以 RAPIDS 和 FAISS-GPU 重组关键加速环节，三是在多 GPU 环境中采用复制式多实例、请求级动态调度与 bounded batching 追求近线性的吞吐扩展。

6.2　后续工作展望

后续研究可从三个方向继续推进。第一，在统一脚本与统一环境下补齐正式实验图表，并对不同数据规模、不同检测器和不同参数区间进行系统复核。第二，在更复杂的多 GPU 与多节点场景下，继续研究复制与分片之间的折中、异构设备协同以及更细粒度的负载反馈机制。第三，将当前框架推广到在线增量更新、风险解释增强与更贴近生产环境的长时间运行场景，从而进一步检验其工程稳健性。

正式版补齐图 6-1

图 6-1　后续研究路线图

图示内容要求：建议按时间线展示“补齐统一脚本实验 - 扩展多数据集与多检测器 - 研究多节点与异构调度 - 推进正式送审版”的四阶段路线。

图示作用说明：图 6-1 用于将本文结论自然过渡到未来工作，增强全文的完成度与延展性。

参考文献

[1] Zhao Y, Chen G, Jia Z. TOD: GPU-accelerated outlier detection via tensor operations[J]. Proceedings of the VLDB Endowment, 2022.

[2] Aggarwal C C. Outlier analysis[M]. 2nd ed. Springer, 2017.

[3] Johnson J, Douze M, Jégou H. Billion-scale similarity search with GPUs[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2019.

[4] Xiao Y, Sun M, Song Z, et al. Breaking the storage-compute bottleneck in billion-scale ANNS: a GPU-driven asynchronous I/O framework[Z]. 2025.

[5] Malkov Y A, Yashunin D A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020.

[6] Crankshaw D, Wang X, Zhou G, et al. Clipper: a low-latency online prediction serving system[C]. Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation. 2017.

[7] Gu J, Lee Y, Zhang C, et al. Clockwork: predictable and efficient deep learning serving for cloud[C]. Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation. 2020.

[8] Blumofe R D, Leiserson C E. Scheduling multithreaded computations by work stealing[J]. Journal of the ACM, 1999.

[9] Moritz P, Nishihara R, Wang S, et al. Ray: a distributed framework for emerging AI applications[C]. Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation. 2018.

[10] 本文项目组. 统一脚本、参数配置、日志模板与送审前复核记录[Z]. 2026.

附录 A　实验脚本与日志模板

本附录在正式版中应补充关键脚本入口、命令行参数模板、日志字段样例与目录组织方式，以支撑“统一脚本、统一日志、统一口径”的研究方法。相关脚本与复核记录可与内部材料统一维护和版本化管理[10]。

A.1　建议补充内容

- 关键训练、建索引、查询与评测脚本入口。
- 常用命令行参数模板与默认值说明。
- 日志字段示例及字段解释。
- 工程目录组织方式与结果归档规范。

附录 B　图表补充说明

本附录应集中说明正文各图表的变量命名、单位、比较口径与补图要求，确保图表在送审版和答辩版中保持一致。

B.1　建议补充内容

- 图表中的变量名称、量纲和取值范围。
- 不同图表之间共享指标的统一定义。
- 同口径比较、跨阶段比较与跨后端比较的边界说明。
- 送审版与答辩版图表的共用与差异化补充规则。

附录 C　缩写、变量与配置对照

本附录列出正文中频繁出现的关键变量、配置项及其含义，便于后续统一实验脚本和结果复核。

变量或配置项	含义
`batch_size`	单次提交到候选召回或精排路径中的批处理大小。
`rerank_budget`	进入高精度精排阶段的候选数量预算。
`nlist`	索引聚类桶数量或粗排分区规模参数。
`nprobe`	查询阶段实际访问的粗排桶数量。
`device_id`	当前任务绑定的 GPU 设备标识。
`replica_mode`	多 GPU 复制式多实例执行模式开关。
`bounded_batch_size`	在时延约束下允许形成的微批上限。

致谢

本版本以论文结构与技术内容定稿为主，致谢内容可在正式送审前根据实际情况补充。